

Logistische Regression

Von der Geraden zur Klassifikation

Die Idee

Lineare Regression sagt *Zahlen* vorher (z. B. einen Preis). Aber was, wenn die Antwort **Ja** oder **Nein** ist – Spam oder kein Spam? Bestanden oder durchgefallen? Ziffer 3 oder nicht? Die **logistische Regression** berechnet stattdessen eine **Wahrscheinlichkeit** zwischen 0 und 1 mithilfe der **Sigmoid-Funktion**: $\sigma(z) = \frac{1}{1+e^{-z}}$. Die Eingabe z ist – wie bei der linearen Regression – eine gewichtete Summe: $z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$.

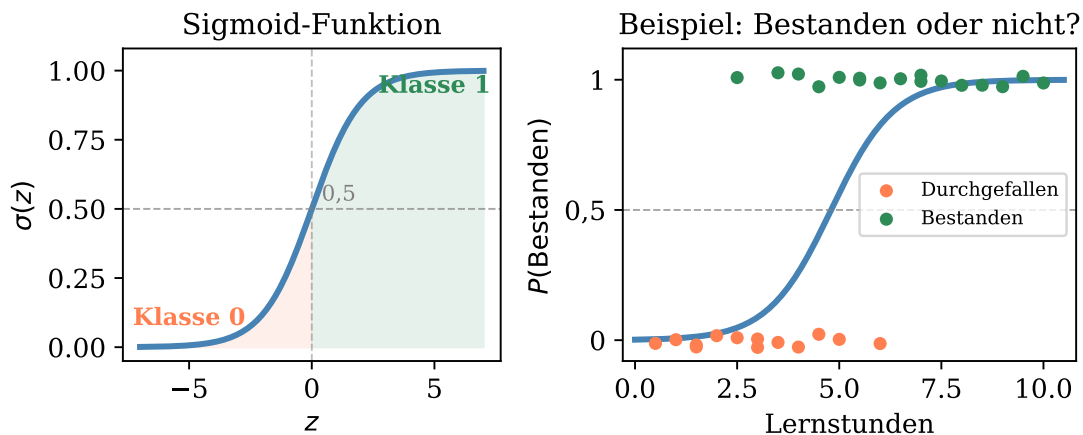


Abbildung 1: **Links:** Die Sigmoid-Funktion bildet jede reelle Zahl auf $[0, 1]$ ab. **Rechts:** Beispiel – je mehr Lernstunden, desto höher die Wahrscheinlichkeit zu bestehen.

Die Entscheidungsregel

Ist $\sigma(z) > 0,5$ (also $z > 0$), ordnet das Modell **Klasse 1** zu, sonst **Klasse 0**. Beim Training werden die Gewichte $w_1, \dots, w_n$ und b so optimiert, dass die **Likelihood** der beobachteten Klassen maximal wird – dasselbe Prinzip wie bei der linearen Regression.

Ausblick: Viele Klassen mit One-vs-Rest

Logistische Regression kann auch mehr als zwei Klassen unterscheiden. Für die Ziffernerkennung (0–9) trainiert man **10 binäre Modelle** – eines pro Ziffer. Jedes beantwortet eine einfache Frage: *Ist es meine Ziffer oder nicht?* Am Ende gewinnt die Ziffer, deren Modell die höchste Wahrscheinlichkeit liefert.

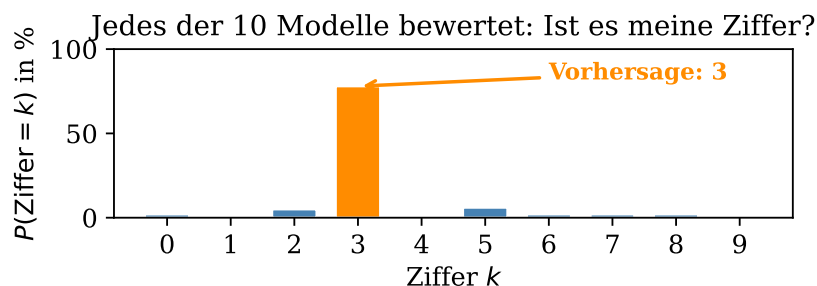


Abbildung 2: One-vs-Rest bei der Ziffernerkennung: 10 Modelle liefern je eine Wahrscheinlichkeit. Die Ziffer mit dem höchsten Wert gewinnt – hier die 3.

Zusammengefasst: Die logistische Regression verwandelt eine gewichtete Summe über die Sigmoid-Funktion in eine Wahrscheinlichkeit. Für zwei Klassen reicht ein einziges Modell; für k Klassen trainiert man k Modelle nach dem *One-vs-Rest*-Prinzip. So lassen sich auch komplexe Aufgaben wie die Ziffernerkennung mit einem konzeptionell einfachen Verfahren lösen.